Session 7 · Deck 7.A2

Nonlinearity: the Activation, XOR, and What a Network Can Approximate

Zhigang Feng

Thursday, November 5, 2026

© Zhigang Feng Redistribution prohibited without permission

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 7.A2 ▼

S1 ✓	S2 ✓	S3 ✓	S4 ✓	S5 ✓	S6 ✓	S7	S8	S9	S10
Sep 24	Oct 1	Oct 8	Oct 15	Oct 22	Oct 29	Nov 5	Nov 12	Nov 19	Dec 3
Intro	HA · OLG	Num. DP	Numerics	Numerics	HA classic	ML	RL	Text	Agents
Growth	Policy	Python	I	II	The wall	Deep nets	HA + DL	LLMs	Projects
trunk	branches	the machine · classical methods			the wall	new tools		new data	colleagues

Build it on paper → solve it on the machine → push it to the frontier with AI

Block A — learning a function: 7.A1 the learning problem and the neuron · **7.A2** activations and approximation · 7.A3 backpropagation and training. **Block B — using it:** 7.B1 PyTorch and a forecast · 7.B2 the growth model · 7.B3 architecture and shape · 7.B4 capacity and order.

© Zhiyuan Feng · Redistribution prohibited without permission

Roadmap: Four Parts

7.A1 left one neuron, $h = \sigma(w^\top x + b)$, and one unexamined piece: the activation σ.

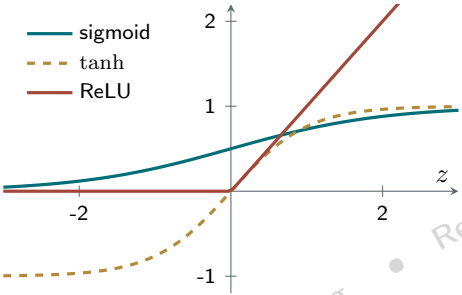
Part	Topic	The question it answers
1	The activation	What is σ, and which ones will you actually meet?
2	Why one layer is not enough	What fails in a layer without σ, and what does a second layer fix that a wider first layer cannot?
3	How a network bends a straight line	How do kinks add up to a curve, and what changes when the kinks are learned?
4	What the theory says	Which functions can the family reach, how fast, and what is never promised?

© Jiaxiang Feng Redistribution prohibited without permission

Idea

Economists already choose flexible functional forms: translog, CES, polynomials, splines — and the Chebyshev basis of 5.B2. A network is one more flexible form. **The difference is that it chooses its own basis.**

Two that saturate, one that does not.

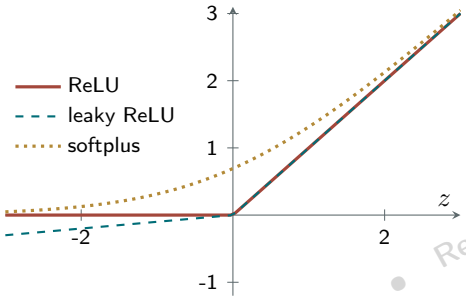


- **Sigmoid**, $1/(1+e^{-z})$, in $(0,1)$; **tanh**, in $(-1,1)$. Both **saturate**: flat in the tails.
- **ReLU**, $\max(0,z)$: zero on one side, the identity on the other. Slope exactly 0 or 1.
- ReLU for hidden layers; sigmoid where an output must be bounded.
- **No activation on a regression output: it would silently bound the prediction.**

© Zhigang Feng
Redistribution prohibited without permission

Miao & Feng, *AI for Economists*, §9.2, Table 9.2.

Each hidden ReLU unit contributes one hinge. A network is a sum of hinges.



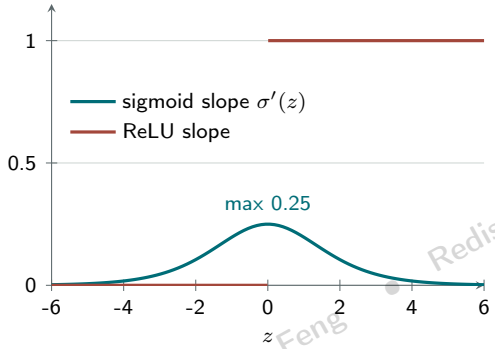
- $\max(0, z)$ is a call option's payoff, and an occasionally binding constraint: inactive, then linear.
- **Leaky ReLU**: a small slope on the flat side, so a unit that switched off can return.
- **Softplus**, $\log(1 + e^z)$, rounds the corner, for models that need second derivatives.
- **Each hidden ReLU unit contributes one hinge. A network is a sum of hinges.**

© Zhigang Feng
Redistribution prohibited without permission

Miao & Feng, *AI for Economists*, §9.2, Fig. 9.4.

1 · Saturation, and Where the Vanishing Gradient Begins

A slope below one, multiplied once per layer, kills the signal; ReLU's slope is one.



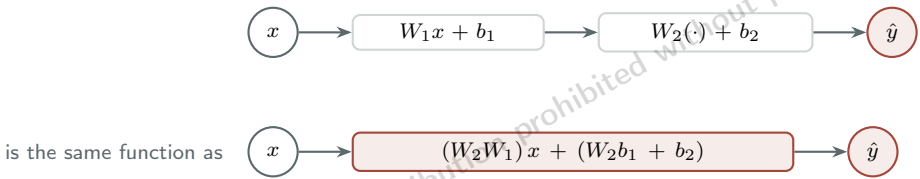
- The sigmoid's slope is $\sigma(z)(1-\sigma(z))$, never above 0.25.
- Training sends an error signal backwards through the layers (7.A3), multiplied by this slope once per layer: at best 0.25^L.
- $L=3$: 0.016. $L=5$: 0.00098. $L=7$: 6×10^{-5}. The early layers barely move: the **vanishing gradient**.
- **ReLU's slope is 1 on its active side, so the signal survives depth. That is why it is the default.**

© Zhigang Feng Redistribution prohibited without permission

Miao & Feng, *AI for Economists*, §9.2.

Linear Stacks Stay Linear

Remove the activation and a hundred layers are one regression line.

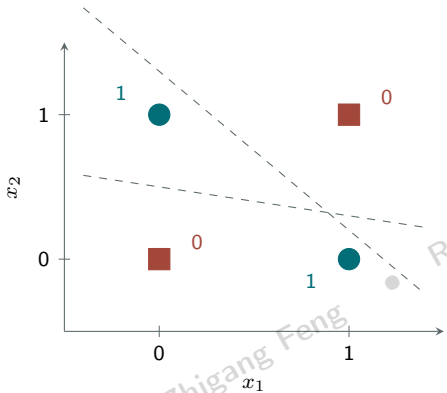


© Zhigang Feng · Redistribution prohibited without permission

- Remove the activation and compose two layers: $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$.
- That is one linear map. A hundred linear layers are still one regression line.
- **Every bit of a network's flexibility comes from the activation between the linear maps.**

Miao & Feng, *AI for Economists*, §9.1.

Exclusive or: no single straight line separates the two classes.



© Zhigang Feng

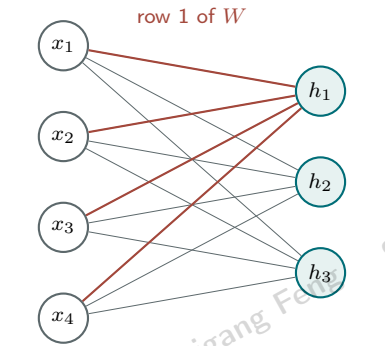
Redistribution prohibited without permission

- **XOR**, exclusive or: the output is 1 when exactly one of two binary inputs is 1.
- The two 1s sit on one diagonal, the two 0s on the other. No straight line separates them: try the dashed ones.
- Minsky and Papert (1969) made this the standard objection to single-layer networks.
- **The cure is a second layer. That is the rest of this part.**

M. Minsky and S. Papert, *Perceptrons*, MIT Press, 1969.

A Layer Is Many Neurons Reading the Same Input

A layer is a system of m regressions sharing their regressors.

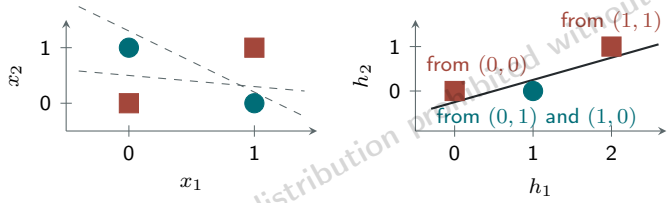


- Put m neurons side by side. Each has its own weights and bias; all read the same x.
- Stack the weight vectors as rows of a matrix W ($m \times n$): $h = \sigma(Wx + b)$, with σ applied entry by entry.
- Here W is 3×4: 12 weights and 3 biases, 15 parameters.
- **A layer is a system of m regressions sharing their regressors.**

© Zhigang Feng · Redistribution prohibited without permission

The Same Four Points, Moved by the Hidden Layer

The hidden layer did not fit a curve — it moved the points until one line was enough.



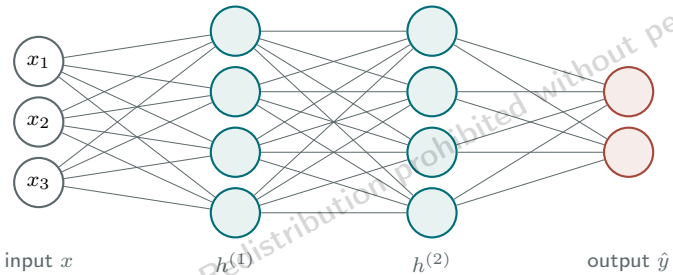
© Zhirang Feng · Redistribution prohibited without permission

- **The same four points, drawn twice.** Same marks, same colours; only the axes change.
- Write $s = x_1 + x_2$. Then $h_1 = \max(0, s)$ and $h_2 = \max(0, s - 1)$: $(0,0) \mapsto (0,0)$, both $(0,1)$ and $(1,0) \mapsto (1,0)$, $(1,1) \mapsto (2,1)$.
- The drawn line is the output $\hat{y} = h_1 - 2h_2$: 0, 1, 1, 0, *exact*, not approximate. **The hidden layer did not fit a curve — it moved the points until one line was enough.**

Miao & Feng, *AI for Economists*, §9.2; Goodfellow, Bengio, Courville, *Deep Learning*, 2016, §6.1.

The Object in One Line

A network is a parametric family: alternate a linear map with a fixed nonlinearity.



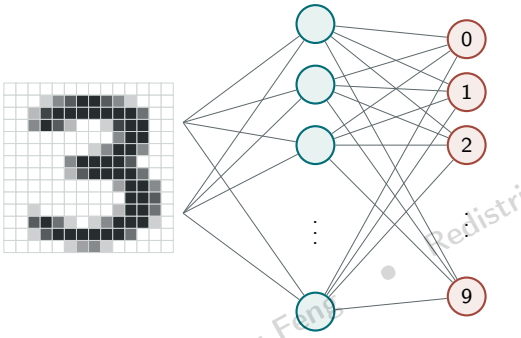
Redistribution prohibited without permission
© Lorraine Fan

- A **feedforward network** (multilayer perceptron, MLP) composes layers: $f_\theta(x) = W_3\,\sigma\big(W_2\,\sigma(W_1 x + b_1) + b_2\big) + b_3$.
- θ collects every W_ℓ, b_ℓ. **Width**: units per layer. **Depth**: number of layers.
- **A network is a parametric family: alternate a linear map with a fixed nonlinearity.**

Miao & Feng, *AI for Economists*, §9.1, eq. (9.6).

A Network for Digits: 784 → Hidden → 10

More parameters than observations, and it still generalises.

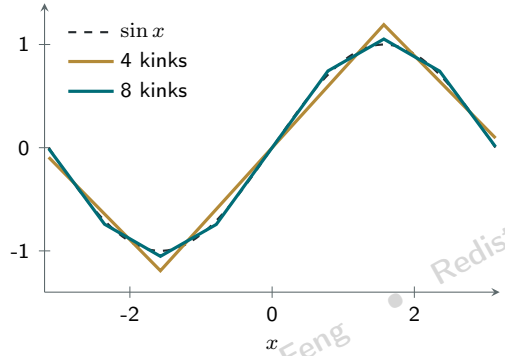


- Layer 1: 784×128 weights $+ 128$ biases $= 100{,}480$ parameters.
- Layer 2: $128 \times 10 + 10 = 1{,}290$. Total: **101,770**.
- Trained as in 7.A3, it classifies 97 to 98% of the 10,000 test images correctly.
- **More parameters than observations, and it still generalises. 7.B4 asks why.**

© Zhigang Feng · Redistribution prohibited without permission

Accuracy: the 784–128–10 example of the TensorFlow beginner tutorial; the same one-hidden-layer design in LeCun et al. (1998), 300 units: 95.3%.

Fix the kinks and the approximation is least squares on a piecewise-linear basis.



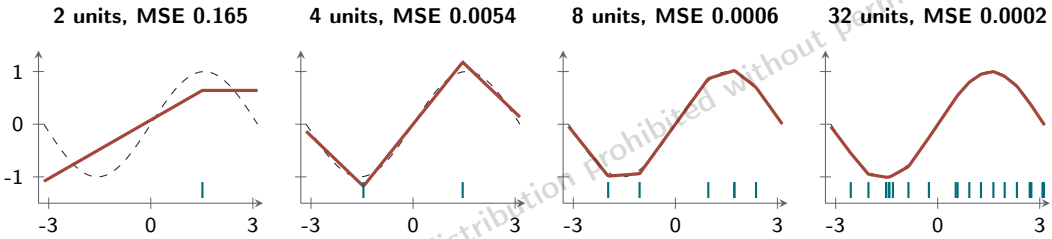
© Zhigang Feng
Redistribution prohibited without permission

- One hidden ReLU layer, one input: $\hat{y}(x) = a_0 + \sum_{i=1}^{N} a_i \max(0,\, x - b_i)$.
- The shifts b_i are the **kinks**, where the slope may change; the a_i set the slope on each interval.
- Fix the b_i, evenly spaced, and $\hat{y}$ is linear in the a_i: ordinary least squares on a piecewise-linear basis — 5.B3's finite elements, in other words.
- **Mean squared error 0.0059 with 4 kinks, 0.0003 with 8. There is no network here yet.**

Lab 7, block B; least squares on 121 points of $\sin x$.

The Turn: Let the Kinks Choose Where to Go

Train the kinks with the coefficients: the approximant decides where to bend.

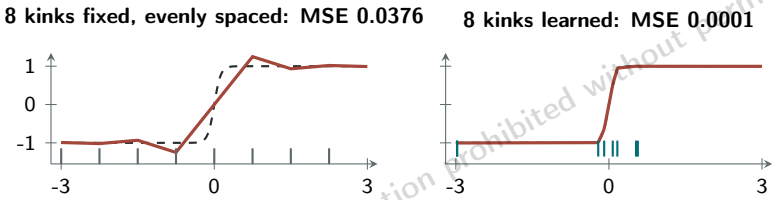


© Zacharia Fadl Redistribution prohibited without permission

- Train the b_i together with the a_i: a one-hidden-layer network. The teal ticks mark the learned kinks, and they are not evenly spaced.
- At 8 units fixed kinks by least squares (0.0003) beat training (0.0006); at 4 units training (0.0054) beat fixed (0.0059). A search, not a solve.
- **The basis is no longer chosen in advance. The approximant decides where to bend.**

Lab 7, block B; Adam, 3,000 steps, one seed.

Where the curvature is concentrated, learned kinks go to it; fixed ones cannot.



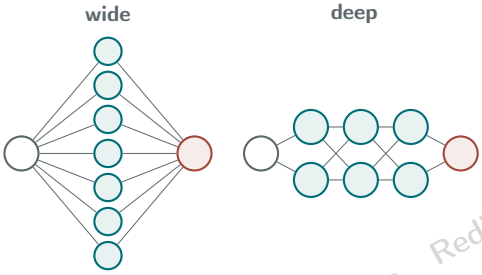
Redistribution prohibited without permission

- $\tanh(8x)$: all the curvature near zero. Eight units either way.
- Six of the eight learned kinks crowd into $[-0.21, 0.57]$, about an eighth of the domain; the fit is **396 times** better. On $\sin x$, whose curvature is spread evenly, training lost.
- A fixed basis is linear in its parameters: a vector space, a closed-form solve, a rate quotable in advance. Width-m networks are none of these.
- **A solve becomes a search. What that buys is priced next.**

© Zhiang Fan

Adam, 6,000 steps, one seed, against least squares on the same eight kinks. Lab 7, block B.

Width adds basis functions; depth composes them.



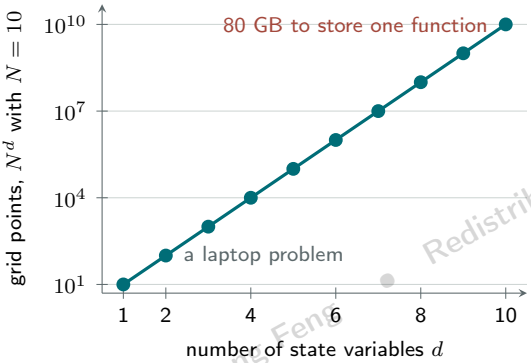
- Width adds basis functions; depth composes them.
- Cost: 20 inputs, three hidden layers of 256 units: $5{,}376 + 65{,}792 + 65{,}792 + 257 = 137{,}217$ parameters.
- Some functions need exponentially many units in two layers, polynomially many in three.
- **Depth pays when the target is a composition: data → beliefs → choice; households → aggregates.**

© Zhigang Feng · Redistribution prohibited without permission

R. Eldan and O. Shamir, "The power of depth for feedforward neural networks", COLT 2016; Miao & Feng, §9.1–9.2.

The Curse of Dimensionality, in Arithmetic

A grid costs N^d. A method whose cost grows with the examples changes what can be solved.



- A grid with N points per state variable needs N^d points in d dimensions.
- With $N = 10$: 100 points at $d = 2$, a million at $d = 6$, ten billion at $d = 10$, which is 80 GB for one policy function.
- Wealth, income, age, housing and two aggregate shocks already make $d = 6$ — 6.B1's Krusell–Smith household already has four, (a, ε, z, K).
- **A method whose cost grows with the number of examples, not with N^d, changes what can be solved.**

© Zhigang Feng · Redistribution prohibited without permission

Universal Approximation, on a Compact Domain

A good network exists on the domain. Nothing is said outside it.

- A **compact domain** is closed and bounded: $[0,1]^d$, or a capital range $[k_{\min}, k_{\max}]$.
- **Universal approximation**: for any continuous f on a compact domain and any $\varepsilon > 0$ there is a one-hidden-layer network $\hat{f}(x) = \sum_{j=1}^{m} v_j\, \sigma(w_j^\top x + b_j) + c$ with $\sup_x |f(x) - \hat{f}(x)| < \varepsilon$.
- Cybenko (1989) for sigmoids; Hornik (1991) for any bounded non-constant activation; Leshno et al. (1993) for any that is not a polynomial.
- Polynomials have the same property (Stone–Weierstrass, the reason 5.B2 can use Chebyshev), so universality does not distinguish a network.
- **The theorem says a good network exists on the domain. It says nothing outside the domain.**

© Zhigang Feng · Redistribution prohibited without permission

Cybenko, *Math. Control Signals Syst.* 2, 1989; Hornik, *Neural Networks* 4, 1991; Leshno, Lin, Pinkus, Schocken, *Neural Networks* 6, 1993.

Four Things the Theorem Does Not Say

Existence is a statement about the family, not about the network you trained.

- **Not how many units.** "There exists m", and m may be astronomically large.
- **Not how to find the weights.** Good weights exist; finding them is a non-convex search that promises only a stationary point.
- **Not how much data.** Approximating a known function is different from estimating an unknown one from a finite sample.
- **Not that it will generalise.** Fitting the sample and fitting the function are different achievements.
- **Finding the weights** is the binding one. With 8 kinks, least squares reached mean squared error 0.0003; training the same 8 units reached 0.0006. A better member of the family existed and the optimiser did not find it.
- **Existence is a statement about the family, not about the network you trained.**

© Zhiguo Feng · Redistribution prohibited without permission

Miao & Feng, *AI for Economists*, §9.1.

Three Errors: Approximation, Estimation, Optimization

Only the sum is observed, and a bigger network moves the three parts in different directions.



- **Approximation error**: the family cannot reach f. Falls with width and depth. The theorems live here.
- **Estimation error**: a finite sample picks the wrong member. Falls with data, rises with flexibility.
- **Optimization error**: the algorithm stops short of the sample optimum. 7.A3.
- **Only the sum is observed, on held-out data. A bigger network lowers the first error and can raise the other two.**

© Zhirang Feng · Redistribution prohibited without permission

L. Bottou and O. Bousquet, "The tradeoffs of large scale learning", NeurIPS 2007.

Barron's Theorem, and the Class It Holds For

No d in the exponent — but the dimension moved into the constant and the class.

- Barron (1993): with m sigmoidal hidden units, on a ball of radius r,

$$\int_{B_r} \left(f - \hat{f}_m\right)^2 d\mu \;\le\; \frac{(2rC_f)^2}{m}, \qquad C_f = \int \|\omega\|\,|\tilde{f}(\omega)|\,d\omega,$$

  where $\tilde{f}$ is the Fourier transform of f.
- The **Barron class** is the set of functions with finite C_f: little weight at high frequencies.
- Double the units and the squared error halves, and the input dimension d is absent from the exponent.
- But C_f is an integral over $\mathbb{R}^d$. For many functions it grows with d; for others it is infinite.
- **The dimension did not vanish. It moved into the constant and into the narrowness of the class.**

© Zhigang Feng · Unauthorized distribution prohibited without permission

A. Barron, "Universal approximation bounds for superpositions of a sigmoidal function", *IEEE Trans. Inf. Theory* 39, 1993.

The Rates, Side by Side

The gap is between a basis fixed before seeing f and a basis adapted to f.

Functions	Approximator, size m	Best possible	Source
k times smooth	any continuous method	$\gtrsim m^{-k/d}$ (error)	DeVore et al. (1989)
Barron class	any basis fixed in advance	$\gtrsim m^{-2/d}$ (squared)	Barron (1993), Thm. 6
Barron class	one hidden layer	$\leq C/m$ (squared)	Barron (1993), Prop. 1
compositional	matching deep network	exponent free of d	Poggio et al. (2017)

- Rows 1–2 are the curse of dimensionality — Chebyshev and Smolyak (5.B2, 5.B3) are bases fixed in advance. Rows 3–4 escape it by assuming something about f.
- **The gap is between a basis fixed before seeing f and a basis adapted to f.**

© Zhaoning Huang • Redistribution prohibited without permission

DeVore, Howard, Micchelli, *Manuscripta Math.* 63, 1989; Poggio et al., *Int. J. Autom. Comput.* 14, 2017.

Takeaways

- **Part 1.** With ReLU, each hidden unit is one hinge and a network is a sum of hinges; a saturating σ multiplies the backward signal by at most 0.25 per layer.
- **Part 2.** Without an activation, depth buys nothing. A hidden layer changes the coordinates until a line is enough; XOR shows this exactly, not approximately.
- **Part 3.** Fix the kinks and it is least squares; free them and the approximant chooses where to bend — $396\times$ better where the curvature is concentrated, worse where it is not.
- **Part 4.** Universal approximation is existence on a compact domain: not size, not a recipe, not data, not generalisation. Barron's rate has no d in the exponent; the dimension lives in the constant and the class.

© Zhigang Feng · Redistribution prohibited without permission

Bridge: From “Exists” to “Found”

Every promise in this deck was “a good network exists”. At 8 kinks we already saw the optimiser stop short of a member of the family that least squares found.

The next deck is how to find one: the gradient of a loss with respect to every weight, computed by the chain rule run backwards; a step against it; and a rule, decided before looking, for when to stop.

Next: 7.A3

Backpropagation by hand, on two networks small enough to check; why the optimizer is first-order (4.A2 already said why); the loss as an assumption about the noise; and three samples, each with one job.

© Zhigang Feng
Redistribution prohibited without permission